SciCapenter-Supporting Caption Composition for Scientific Figures with Machine-Generated Captions and Ratings带读

SciCapenter：Supporting Caption Composition for Scientific Figures with Machine-Generated Captions and Ratings：把“生成 + 评分 + 检查表 + 上下文”打包成写作助理，显著降低写作者认知负担

0. TL;DR（≤120字）

- 贡献一句话：把机器生成caption、质量评分（rating）、要素检查表（checklist）和引用段落（referred paragraphs）整合进同一写作界面，支持迭代改写。
- 适用一句话：当“生成文本未必更好，但能当草稿/诊断工具”时，用“分析信号”帮助人更快写出可用版本。
- 风险一句话：生成caption会出现不准与幻觉（hallucination），且论文主实验没直接证明“最终质量变好”。

---

1. 背景与动机：他们到底在“卡什么”

任务背景：用 5~10 句讲清语境

科学论文里的图（figure）往往承载核心信息，但读者如果没有caption辅助，理解与记忆会明显变差；现实中作者却经常“随手写两句”，质量参差。与此同时，围绕科学图与caption的AI能力已经很丰富：能生成caption、分析图中信息、甚至评估caption有用性（usefulness）。问题是：这些技术大多在“读者视角”被评测（让人看机器产物打分），而不是在“写作者视角”验证：它们到底能不能让人更快、更省脑子地写出东西。

痛点拆解：为什么已有方法不够

- 写作任务的本质不是“产出一段完美文本”，而是“在有限时间里，凑齐关键要素、减少遗漏、控制篇幅、与正文一致”。单纯把机器生成caption丢给作者，往往不够可控，也不够“可诊断”。
- 生成caption常出现不精确甚至幻觉（hallucination）；短caption尤其容易跑偏，因此“生成即答案”的路径很危险。
- 现有一些混合式系统（mixed-initiative）做过可视化标题/说明生成，但没有专注“学术论文 figure caption 组合写作”的场景化需求与工作流（PDF里定位图、看正文提到图的段落、反复修改）。

作者的关键洞见：一句话点题

> 就算机器生成文本不更好，只要把“上下文 + 诊断信号（评分/检查表）”一起给到位，也能显著帮人写作。

---

2. 问题定义与符号（足够让人复现）

输入/输出：清晰列出

系统输入（Input）

1. 一篇论文的 PDF（含多张figure与其caption）。
2. 用户选择目标 figure，并在编辑框里写/改 caption。

系统输出（Output）（围绕同一目标figure）

- 图像展示、当前caption、以及正文中提到该图的段落（figure-mentioning / referred paragraphs）。

- 两条机器生成caption：长（long）与短（short）。
- 质量评分（rating，星级）与解释（explanation）。
- 检查表（check table）：标注caption是否覆盖若干关键方面（aspects）。

目标函数/评价指标：必要时给公式

论文把“成功”拆成两层：

1. **写作过程层面的收益**
   主指标是 NASA Task Load Index（NASA TLX，用来量化工作负荷/认知负担）。他们比较“无系统 vs 有系统”，在自由写作与限时写作两种场景下，TLX整体指数都有下降，且限时下降更明显。
2. **组件主观满意度**
   让参与者分别评价 check table、rating、解释、长/短生成caption、referred paragraphs 等组件的有用性/满意度（Likert量表）。结果显示：check table 与 referred paragraphs 最受欢迎，长/短生成caption评分最低。

> 备注：论文主实验没有直接做“最终caption质量显著提升”的大规模专家评测（成本高），在局限性里明确承认。

关键假设：作者默认了什么

- **假设A**：作者写caption时，最缺的是“对齐正文语境 + 避免要素遗漏”，而不仅是“灵感”。所以系统强调提取正文里提到figure的段落，并给检查表/评分做诊断。
- **假设B**：机器生成可能会错，但“可编辑 + 可复评（resubmit for revised evaluations）”能让人把错修回来，收益来自迭代回路（iteration loop）。
- **假设C**：对“写作者”而言，分析信号（labels/ratings）可能比完整生成文本更值钱。

---

3. 方法总览：一张“脑内结构图”

把 SciCapenter 当成一个“写作IDE（integrated development environment）”：它不保证一键写完，而是把你写作时需要的证据、草稿、诊断全塞到一个闭环里。

整体流水线（ASCII）

```
PDF 上传
  │
  ├─(1) 解析 PDF：抽取 figures / captions / abstract / 正文
  │        │
  │        └─(2) 找到每个 figure 的引用段落（referred paragraphs）
选择目标 figure
  │
  ├─(3) 生成两版草稿 caption：Short / Long（摘要式生成）
  │
  ├─(4) caption 诊断信号 A：检查表（6个方面 multi-label）
  │
  ├─(5) caption 诊断信号 B：GPT 打分 + 解释（usefulness）
  │
  └─(6) 用户编辑 → 提交 → 重新打分/更新诊断 → 迭代
```

每一步“作用”和“替代方案”

1. **PDF解析**：自动定位所有figure并串起工作流，减少人工复制粘贴。
   - 替代：若有LaTeX源码/结构化源文件，解析更稳，但通用性差。

2. **referred paragraphs（引用段落）**：给写作者“正文证据”，避免caption与正文脱节；也是后续评分/生成模型的重要条件输入。
   - 替代：更精细的结构化解析（section/paragraph-level parsing），但工程成本更高。
3. **短/长两版生成caption**：同一内容两种长度偏好：长更信息量，短更省版面。
   - 替代：不做长短，改成“literal vs explanatory（直译 vs 解释型）”风格切换，更贴近写作意图。
4. **检查表（checklist）**：把“好caption应包含什么”显式化成可见信号。
   - 替代：直接让LLM给建议，但成本更高且一致性不稳定。
5. **评分+解释（rating+explanation）**：把“有用性（usefulness）”变成可迭代反馈通道。
   - 替代：训练本地 quality estimation（QE）模型做评分。
6. **迭代回路**：核心不是一次生成，而是“编辑—反馈—再编辑”。
   - 替代：批量生成多候选+rerank，但对人类写作不如“可解释反馈”顺手。

---

4. 核心方法逐段讲解（按论文逻辑走）

4.1 模块 A：PDF解析与上下文对齐（Document ingestion & grounding）

它做什么：用户丢PDF进来，系统抽取figure/caption，并把“正文里提到该图的段落”抓出来，供写作者参考。
直觉：写caption最怕“我以为图表达X，但正文其实在讨论Y”。referred paragraphs 就是对齐锚点（grounding anchor）。
形式化（伪代码）

```
def process_pdf(pdf_bytes): doc_id = mongodb.save(pdf_bytes) # store raw file figures = pdffigure2.extract(pdf_bytes) # figures, captions, etc. full_text = pdffigure2.extract_text(pdf_bytes) for fig in figures: fig.referred_paragraphs = regex_find_paragraphs(full_text, fig.index) return figures
```

工程细节（实现坑、默认超参、复杂度）

- PDF解析跨领域不稳定：排版差异会导致抽取失败，影响用户研究中的数据准备。
- 论文研究里排除了表格（tables），因为表与figure差异大；提醒你不要轻易扩大系统边界。

4.2 模块 B：两路机器生成caption（Short/Long generation）

它做什么：给目标figure生成两条候选caption：长版与短版。生成模型不是看图像，而是“摘要式（summarising）”利用 figure-mentioning paragraphs 写caption。
直觉

- 机器生成不一定更好，但作为“起步草稿”能把作者推入编辑状态。
- 长/短并存对齐真实写作矛盾：信息量 vs 篇幅限制。
  形式化（关键点）
- 复用 Pegasus+ / Pegasus++ 系列：输入为引用段落，输出为caption。
- Pegasus++ 用“>30词长caption”训练，因此更偏向生成长caption。
- 推理使用 beam search，常见设定 num_beams=5。
  工程细节
- 生成caption会不准甚至 hallucination；短caption更容易跑偏。工程上最好把生成视为“可编辑草稿”，而不是最终答案。
- 因此必须配套“诊断信号”（检查表与评分解释），否则生成会反噬用户信任。

4.3 模块 C：检查表（Check table）——把caption拆成6个可见维度

它做什么：判断当前caption是否覆盖6个方面（aspects），缺失就提示。六个方面：

- Helpfulness（有用性）
- OCR mention（是否提到图中出现的词/短语；OCR=Optical Character Recognition）
- Relation（是否描述元素间关系：更高/更低/最大/最小等）
- Stats（是否提到数字/统计量）
- Takeaway（是否写出高层结论/要点）
- Visual（是否提到颜色/形状/位置等视觉属性）

直觉

- 写作最难的一步常是“我到底漏了什么？”检查表把隐性标准显性化，像编译器warning一样提醒你。
- 实证上，参与者往往更喜欢 check table 与 referred paragraphs，而不是生成caption本身。

形式化

- 用 SciBERT 微调为多标签（multi-label）分类器：输入caption文本，输出6维0/1。
- 训练数据约 3,159 条标注，性能约 F1=0.64。

伪代码：

```
aspects = scibert_multilabel.predict(caption_text) # aspects: dict
{helpfulness:0/1, ocr:0/1, relation:0/1, stats:0/1, takeaway:0/1, visual:0/1}
```

工程细节（实现坑）

- F1=0.64 说明这不是“铁判官”，只能当提示信号；过度依赖会误导写作者。
- 并非每个caption都需要覆盖所有方面；用户反馈里也出现“强行要求stats不合适”等情况。
- 论文在pilot后把提示从“强制/惩罚式”改成更温和的警示符号，避免制造压力与误用。

4.4 模块 D：评分 + 解释（Caption rating & explanation）

它做什么：用 LLM 对caption在给定段落上下文下做“usefulness”打分（1~6），并生成解释。
直觉

- 对写作者而言，“你哪里不行”比“给你一段新文本”更有用，尤其在你已经有初稿之后。
- 解释能降低“黑箱分数”的挫败感。
  形式化（prompt骨架）

```
Given the paragraph and caption below, please rate the level of usefulness ...
Paragraph: "[paragraph]" Caption: "[caption]"
```

工程细节

- 为控制交互延迟（latency），论文把 GPT-4 换成更快的模型版本（核心信息：速度是可用性的硬门槛）。
- 为防止“刷分/骗分”（Goodhart’s law 风险），限制每个任务的提交次数（例如最多两次请求评分）。

---

5. 训练/推理细节（工程读者最关心的部分）

数据处理：tokenizer/归一化/噪声处理等

论文系统侧的预处理重点在“PDF→结构化”：

- pdffigure2 抽取 figure/caption/abstract/正文文本。
- 用正则表达式按 figure index 找 figure-mentioning / referred paragraphs。

复现关键点：

- 段落切分（paragraph segmentation）要稳定，否则“引用段落”会断裂。
- figure编号格式多样（Fig. 3 / Figure 3 / Figure3），正则要覆盖常见变体。

训练策略：batching, lr, schedule, 正则, dropout...

真正训练的模块主要是 check table 分类器（SciBERT 多标签微调）。论文未展开具体超参细节（如 lr、batch size、epoch）——若要精确复现，需要查他们引用的caption evaluation训练设置或补充材料。

推理策略：beam, length penalty, constraints...

- 生成caption：beam search（常见设置 num_beams=5）。
- 长caption倾向主要来自训练分布（Pegasus++以长caption训练），而非推理阶段强行施加 length penalty。

复杂度与资源：显存/时间/吞吐

论文未给出显存/吞吐数字，但工程取舍明确：

- LLM评分模块必须低延迟，否则交互不可用。
- PDF解析失败率会显著影响体验，且跨领域差异明显。

---

6. 实验设计：他们怎么证明“不是玄学”

数据集与划分：重点：是否有泄漏风险、group split 等

这是HCI用户研究而非纯ML benchmark：

- 参与者：STEM PhD学生（跨专业），含pilot与主实验两阶段。
- 每位参与者提供多篇自己领域论文；研究者选若干“原caption评分最低”的图，遮掉原caption让参与者写。
- 通过“参与者略懂主题但未深入读过该论文”的设定，减少“记得原文怎么写”的偏置（降低泄漏/记忆效应）。

Baseline：有哪些对照

四个条件（within-subject）：

1. 自由写作，无系统
2. 自由写作，有系统（并限制评分请求次数）
3. 限时写作，无系统
4. 限时写作，有系统

消融：哪几个结论最关键

严格意义不是模型消融，而是“组件贡献分解”：

- 对 check table、rating、explanation、long/short caption、referred paragraphs 分别做主观评分，找出谁在真实帮助写作。

结果解读：不仅报数，还解释趋势

核心趋势：

- NASA TLX下降，尤其限时场景下降更明显。

- 挫败感（frustration）降幅最大：说明系统在“降低不确定性与压力”方面非常有效。
- “分析信号”类组件（check table、referred paragraphs）比“生成文本”更受欢迎：当生成质量不稳时，诊断与锚定比“替你写”更值钱。

失败案例：若论文没有则推断并标注为推断

论文明确提到生成caption会出现 hallucination，短caption更严重。
【推断】此外，若引用段落本身对figure描述贫弱，referred paragraphs 会变成噪声锚点，甚至误导写作者。

---

7. 批判性复盘：这篇论文哪些地方可能不靠谱

强依赖的前提

- 强依赖 PDF 结构解析质量；跨领域解析失败会直接毁掉体验。
- 强依赖 referred paragraphs 具备足够信息密度；对“正文写得很省略”的论文效果会打折。

可能的负迁移点

- 6维检查表是“科学图caption”的经验归纳，不一定适合所有写作对象；强行套模板会带来冗余信息或错误激励（用户反馈支持这一点）。

复现实验最容易踩的坑

- HCI实验存在顺序效应（learning effect）与评价偏差；虽然做了顺序控制，但难以彻底消除。
- 评分指标可被优化（Goodhart’s law）：需要限制提交次数或做抗刷策略，否则会出现“为分数写caption”的反直觉行为。

与相邻方法的边界（什么时候别用它）

- 如果场景要求“自动端到端产出高可信文本”，而不是“人类在环（human-in-the-loop）写作”，SciCapenter式设计价值会变小。
- 论文也承认：主实验没验证最终caption质量显著提升；若你只关心最终质量排行榜，这篇证据链不够硬。

---

8. 迁移到 Deep Past（Akkadian→English）的落地方案

Deep Past 是 **Akkadian transliteration → English translation** 的纯文本生成/翻译任务。它不像 figure caption 有图像输入，但同样存在：

1. 输出质量不稳定（专名/数字/罕见词形）；
2. 评估指标（BLEU/chrF++）可能被“刷形状”；
3. 需要快速定位错误来源（分词、长度、实体、语法、复读等）。

可迁移的核心不是“Pegasus写caption”，而是这句工程哲学：

> 生成不稳时，把系统重心放到：上下文锚定（grounding）+ 诊断信号（checklist/rating）+ 可执行改写建议（actionable feedback）。

我们的 pipeline 插入点：tokenizer / model / training / decoding / postprocess / eval

把 SciCapenter 的组件映射成 Deep Past 的“训练与调参仪表盘（dashboard）+ reranker”：

SciCapenter 组件	Deep Past 对应物
referred paragraphs（上下文）	同一tablet/文本块的邻近行、元数据（体裁/时代/地名人名表）、或构造的检索上下文（retrieved context）
check table（6维诊断）	translation checklist：实体/数字保真（fidelity）、专名一致性（consistency）、关键术语覆盖（terminology）、流畅度（fluency）、长度充分性（adequacy）、罕见token复制率（copy ratio）
rating + explanation（LLM评价）	离线 QE（quality estimation）模型或规则+小模型综合评分；解释用“错误类型标签（error tags）”代替长解释
long/short caption drafts	两种译风候选：更直译（literal） vs 更通顺（fluent），或 beam=1 vs beam=K 的 n-best

最小可复现实验（MVP）

目标：在 8GB 约束下先吃到“诊断信号+rerank”的收益。

对照组设置

1. Baseline：你们当前最稳的 seq2seq（Transformer/mT5/Marian/NLLB等）+ 标准beam
2. +Checklist Features：对每个候选译文提取“检查表特征”，训练轻量 reranker（线性模型/LightGBM/小MLP）预测 chrF++ 或 pairwise ranking
3. +Context retrieval（可选）：若数据结构允许，把邻近行/检索上下文拼到输入（注意不泄漏标签）

需要改动的代码/配置点（按模块列）

- Tokenizer：加入“transliteration 特殊符号/分隔符”的规范化（点号、连字符、大小写、破损标记等）。
- Decoding：输出 n-best（beam=5）而不是只取 top-1，为 rerank 提供候选池（对应“多候选”的设计）。
- Checklist 特征（cheap、离线）：
  - 数字一致性：源串数字是否在译文中保留（允许格式变化）
  - 专名/术语一致性：基于词表或规则的命中率
  - 复制率（copy ratio）：源token在译文中被保留的比例（对罕见符号很重要）
  - 长度比：len(hyp)/len(src)
  - 重复惩罚：n-gram重复率（避免循环式胡言乱语）
- Reranker：用 dev 划分训练；强烈建议按 tablet/text 来源做 group split，避免同源泄漏导致虚高。

验收指标

- 主指标：chrF++（主赛指标优先） + BLEU（辅助对照）
- 额外诊断指标（必须记录）：长度分布、copy ratio 分布、数字错误率、专名错误率、稀有token覆盖率

预计算力（8GB 场景）与降级方案

- 8GB最稳路线：不引入大QE模型，先用“规则特征 + 轻量reranker”提升稳定性。
- 若能稍上强度：加一个小型encoder做 QE 分类/排序，但严格控制序列长度与batch。
- 避免在线LLM评分：比赛环境通常不允许外部调用；评分器必须本地化。

进一步迭代路线（最多 5 条，按优先级）

1. 把检查表从“后验诊断”升级到“训练时多任务（multi-task）”：共享encoder，加辅助头预测数字/专名保真等。
2. n-best rerank 正式化：从回归chrF++改为 pairwise ranking，更稳。
3. 模板化后处理（postprocess templates）：对数字/括号/固定符号做一致化，减少低级错误。
4. 检索增强（RAG）：从训练集挖相似片段做提示拼接（严格防泄漏）。
5. 错误分析仪表盘：每条样本输出 checklist、候选对比、错误类型统计；提升团队迭代速度。

9. 读完该做什么（行动清单）

立刻能做的 3 件事（1 小时内）

1. 把 baseline 改成输出 n-best（beam=5）并保存候选池。
2. 写一个最小 checklist 特征脚本：长度比、数字一致性、重复率、copy ratio；把特征与 chrF++ 对齐存表。
3. 在 dev 上快速验证：用线性模型/LightGBM做 rerank，看 chrF++ 是否稳定上涨。

需要半天的 3 件事

1. 设计并实现 group split（按文本来源/平板/文档ID），并做泄漏检查清单。
2. 产出“错误类型分布报告”：数字错、专名错、长度异常、重复异常分别占比。
3. 设计“解释输出格式”：不用长自然语言，直接输出 error tags + 对不上token的位置，便于批量排查。

需要 1~3 天验证的 3 件事

1. 多任务头（aux heads）可行性：加入 2~3 个最关键 checklist 约束，评估是否提升而不伤主任务。
2. pairwise reranking：用 n-best 构造训练对（A优于B），比较与回归chrF++的稳定性。
3. 检索增强是否有效：先在训练集内检索拼接提示，在 dev 上严格评估，防过拟合与泄漏。